

Chapitre 8 – Transactions

I.	Définition et objectif	1
II.	Gérer les transactions	1
1.	Begin Transaction	1
2.	Commit	1
3.	Rollback.....	1
4.	Exemples	1
III.	Utilisation avec l'application Compte	2
1.	Démarche à respecter.....	4
2.	Solution.....	4

I. Définition et objectif

Une transaction est une unité **indivisible** de travail.

Si une transaction **aboutit**, toutes les modifications de données apportées lors de la transaction sont **validées** et intégrées de façon permanente à la base de données.

Si une transaction rencontre des erreurs et doit être **annulée** ou restaurée, toutes les **modifications de données sont supprimées**.

Une transaction permet donc de réaliser plusieurs requêtes SQL sous la forme d'un tout, qui réussit ou pas.

II. Gérer les transactions

Une transaction est gérée par programmation, à l'aide de 3 instructions :

- BEGIN TRANSACTION
- COMMIT
- ROLLBACK

1. Begin Transaction

Avant que *Begin Transaction* soit exécuté, chaque commande SQL qui la précède est définitivement exécutée.

Un nom peut être ajouté après, de même qu'un ;

2. Commit

Commit provoque l'enregistrement définitif des modifications sur la base.

Commit peut être suivi de *Transaction* puis éventuellement du nom de la transaction.

3. Rollback

Rollback est le moyen d'annuler tous les changements intervenus depuis le début de la transaction.

Rollback comme pour *Commit* peut être suivi de *Transaction* puis éventuellement du nom de la transaction.

4. Exemples

Avec la base *EpokaAbsences* :

```
begin transaction

insert into Motif (MotLibelle) values ('Mariage');
insert into Motif (MotLibelle) values ('Décès');

rollback
```

Table *Motif* avant :

	MotNo	MotLibelle
▶	1	Maladie
	2	Maternité
	3	Congés annuels
⊗	NULL	NULL

Table *Motif* après

	MotNo	MotLibelle
▶	1	Maladie
	2	Maternité
	3	Congés annuels
⊗	NULL	NULL

La fenêtre exécution indique la bonne exécution :

```
(1 row(s) affected)

(1 row(s) affected)
```

Autre exemple, avec la même base :

```
begin transaction

insert into Motif (MotLibelle) values ('Mariage');

commit

begin transaction

insert into Motif (MotLibelle) values ('Décès');

rollback
```

Le message d'exécution est le même :

```
(1 row(s) affected)

(1 row(s) affected)
```

Mais le résultat dans la table *Motif* différent :

	MotNo	MotLibelle
▶	1	Maladie
	2	Maternité
	3	Congés annuels
	1009	Mariage
⊗	NULL	NULL

III. Utilisation avec l'application Compte

Cette application sert à gérer les comptes utilisés pour les photocopies. Un crédit initial est affecté pour chaque utilisateur, puis chaque photocopie ajoute une consommation faisant diminuer ce crédit.

L'aspect consommation en lien direct avec l'appareil de photocopie n'est pas l'objet de l'étude, et ne sera pas abordé.

L'application réalisée en C# s'appuie sur une base de données SQL Server dont voici le schéma :



La table *Mouvement* conserve le montant (débité si <0, crédité si >0), ainsi que l'intitulé (crédit initial, tirage 20 exemplaires, ...) et la date du jour du mouvement.

D'un point de vue pratique, chaque identifiant de compte (*CptId*) est associé à un badge magnétique qui est reconnu par le photocopieur.

Bâtie selon le modèle de conception (*pattern design*) DAO, l'application est en cours de réalisation. Très voisine de l'application de gestion des absences, deux onglets ont été développés dans cette première phase :

- La gestion simple des comptes (opérations CRUD), avec calcul d'un solde :

Id	Nom	Prénom	Solde
1	DUBONNET	Jean	400
2	LAFORET	Pierrette	1000
3	DUPUIS	Paule	600
4	MORESTEL	Jacques	0

En dessous du tableau, il y a des champs de saisie pour le Nom et le Prénom, et quatre boutons : Ajouter, Modifier, Supprimer, et Mouvements.

- La liste des mouvements affectant un compte (consultation seulement) :

Compte : Jean DUBONNET

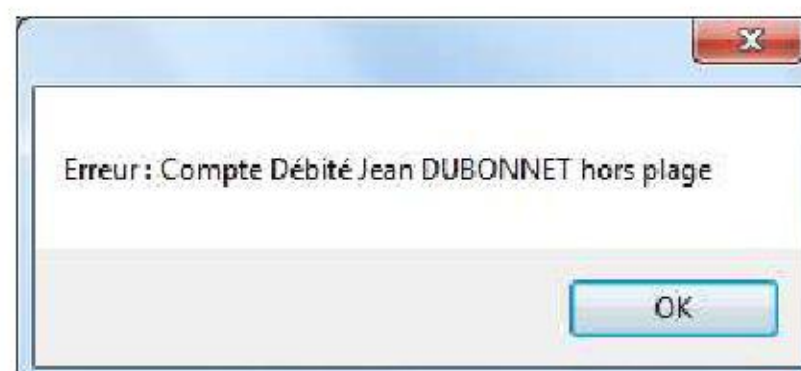
Date	Montant	Libellé
mer. 01/06/2016	1000	Solde initial
mar. 14/06/2016	-600	Echange

Le cahier des charges prévoit une deuxième phase proposant un onglet supplémentaire qui permettra d'effectuer un transfert d'un compte vers un autre. Ce transfert devra donc générer deux mouvements, avec des montants opposés, la même date et le même libellé. Voici la maquette envisagée :

La maquette est une fenêtre intitulée "Gestion des comptes". Elle contient trois onglets : "Comptes", "Mouvements d'un compte" (sélectionné) et "Transfert". Dans l'onglet "Transfert", il y a deux listes déroulantes : "Compte à débiter" avec "Jean DUBONNET" sélectionné, et "Compte à créditer" avec "Pierrette LAFORET" sélectionné. En dessous, il y a un champ "Montant" avec la valeur "50" et un champ "Libellé" avec la valeur "Dépannage". Un bouton "Transférer" est situé en bas à gauche.

Quelques contraintes sont à respecter :

- Le bouton *Transférer* ne doit être actif que lorsque toutes les données sont correctement saisies ;
- Le montant est librement saisissable, mais une données non numérique ou négative doit empêcher le transfert ;
- Les comptes à débiter et à créditer doivent être différents ;
- Il faut qu'à tout moment le solde de chaque compte soit compris entre 0 et 1000. Si tel n'est pas le cas, toute modification doit être annulée et un message affiché, du genre de celui-ci :



1. Démarche à respecter

- Récupérer l'application développée dans sa phase 1 ;
- Créer la base de données *Epoka_Compte* à l'aide des scripts fournis ;
- Définir une méthode *ajouter* dans la classe *MouvementDAO*, prenant en paramètres 2 mouvements à insérer ;
- Appeler cette méthode *ajouter* dans *bTransférer_Click*, après avoir :
 - Créé les deux comptes concernés (classes métiers) via *compteDAO.lireCompte* (méthode à créer) ;
 - Créé les deux mouvements opposés (classes métier) ;
- Effacer ensuite les champs saisis ;
- Mettre à jour enfin les listes (de comptes et de mouvements) ;

2. Solution

Méthode *ajouter* de *MouvementDAO* :


```

public void ajouter(Mouvement unMouvementDebit, Mouvement unMouvementCredit)
{
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    SqlCommand sqlCmd = new SqlCommand("", accesSGBD.connexion);
    try
    {
        sqlCmd.CommandText = "BEGIN TRANSACTION";
        sqlCmd.ExecuteNonQuery();

        // Ajouter le mouvement débit
        sqlCmd.CommandText = "INSERT INTO Mouvement (MvtMt,MvtIntitule,MvtDate,MvtIdCompte) VALUES ("
            + unMouvementDebit.mt.ToString() + ", '" + unMouvementDebit.intitule + "', '"
            + unMouvementDebit.date.ToString("yyyy-MM-dd") + "',"
            + unMouvementDebit.compte.id.ToString() + ")";
        sqlCmd.ExecuteNonQuery();

        // Contrôler que le solde est dans la fourchette 0-1000
        sqlCmd.CommandText = "SELECT sum(MvtMt) as Solde FROM Mouvement "
            + "WHERE MvtIdCompte=" + unMouvementDebit.compte.id.ToString();
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();

        if (sqlDataReader.Read())
        {
            try
            {
                double solde = (double)sqlDataReader["Solde"];
                if (solde < 0 | solde > 1000)
                {
                    throw new ArgumentException("Compte Débité " + unMouvementDebit.compte.nom + " "
                        + unMouvementDebit.compte.prenom + " hors plage");
                }
            }
            finally
            {
                sqlDataReader.Close();
            }
        }

        // Ajouter le mouvement crédit
        sqlCmd.CommandText = "INSERT INTO Mouvement (MvtMt,MvtIntitule,MvtDate,MvtIdCompte) VALUES ("
            + unMouvementCredit.mt.ToString() + ", '" + unMouvementCredit.intitule + "', '"
            + unMouvementCredit.date.ToString("yyyy-MM-dd") + "',"
            + unMouvementCredit.compte.id.ToString() + ")";
        sqlCmd.ExecuteNonQuery();

        // Contrôler que le solde est dans la fourchette 0-1000
        sqlCmd.CommandText = "SELECT sum(MvtMt) as Solde FROM Mouvement "
            + "WHERE MvtIdCompte=" + unMouvementCredit.compte.id.ToString();
        sqlDataReader = sqlCmd.ExecuteReader();
    }
}

```

```

        if (sqlDataReader.Read())
        {
            try
            {
                double solde = (double)sqlDataReader["Solde"];
                if (solde < 0 | solde > 1000)
                {
                    throw new ArgumentException("Compte Crédité " + unMouvementCredit.compte.nom + " "
                        + unMouvementCredit.compte.prenom + " hors plage");
                }
            }
            finally
            {
                sqlDataReader.Close();
            }
        }

        // Valider la transaction
        sqlCmd.CommandText = "COMMIT";
        sqlCmd.ExecuteNonQuery();
    }
    catch (Exception ex)
    {
        sqlCmd.CommandText = "ROLLBACK";
        sqlCmd.ExecuteNonQuery();
        System.Windows.Forms.MessageBox.Show("Erreur : " + ex.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
}

```

Méthode *lireCompte* de *CompteDAO* :


```

public Compte lireCompte(int id)
{
    Compte unCompte = null;
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT CptId,CptNom,CptPrenom,sum(MvtMt) as Solde "
            + "FROM Compte left join Mouvement on CptId=MvtIdCompte where CptId=" + id.ToString()
            + " group by CptId,CptNom,CptPrenom", accesSGBD.connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            if (sqlDataReader.Read())
            {
                unCompte = new Compte();
                unCompte.id = (int)sqlDataReader["CptId"];
                unCompte.nom = (String)sqlDataReader["CptNom"];
                unCompte.prenom = (String)sqlDataReader["CptPrenom"];
                if (!sqlDataReader.IsDBNull(3))
                {
                    unCompte.solde = (double)sqlDataReader["Solde"];
                }
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (Exception sqlEx)
    {
        System.Windows.Forms.MessageBox.Show("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
    return unCompte;
}

```

Méthode *bTransferer_Click* de l'interface :

```

private void bTransferer_Click(object sender, EventArgs e)
{
    try
    {
        CompteDAO compteDAO = new CompteDAO();
        int noCompteDebite = int.Parse((cbCompteDebite.SelectedItem as ComboBoxItem).code.ToString());
        Compte unCompteDebite = compteDAO.lireCompte (noCompteDebite);
        int noCompteCredite = int.Parse((cbCompteCredite.SelectedItem as ComboBoxItem).code.ToString());
        Compte unCompteCredite = compteDAO.lireCompte (noCompteCredite);

        MouvementDAO mouvementDAO = new MouvementDAO();
        Mouvement unMouvementDebit = new Mouvement();
        Mouvement unMouvementCredit = new Mouvement();

        unMouvementDebit.mt=-float.Parse (tbMt.Text.ToString());
        unMouvementDebit.intitule = tbLibelle.Text.ToString();
        unMouvementDebit.date = DateTime.Now;
        unMouvementDebit.compte = unCompteDebite;

        unMouvementCredit.mt = float.Parse(tbMt.Text.ToString());
        unMouvementCredit.intitule = tbLibelle.Text.ToString();
        unMouvementCredit.date = DateTime.Now;
        unMouvementCredit.compte = unCompteCredite;

        mouvementDAO.ajouter(unMouvementDebit, unMouvementCredit);

        cbCompteDebite.SelectedItem = null; // Remise à blanc des zones saisies
        cbCompteCredite.SelectedItem = null;
        tbMt.Text = "";
        tbLibelle.Text = "";

        majListeComptes(); // Les soldes sont à recalculer
        majListeMouvements(); // Des mouvements ont été ajoutés
    }
    catch (Exception ex) {
        System.Windows.Forms.MessageBox.Show("Erreur : " + ex.Message);
    }
}

```